

EDA Notebook

0. Setup Environment

```
In [1]: # DO NOT MODIFY THE CODE IN THIS CELL
!pip install -q utstd

from utstd.folders import *
from utstd.ipyrenders import *

at = AtFolder(
    course_code=36106,
    assignment="AT3",
)
at.run()

import warnings
warnings.simplefilter(action='ignore')
```

```
12.9/12.9 MB 38.4 MB/s eta 0:00:00
4.9/4.9 MB 36.4 MB/s eta 0:00:00
```

ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the source of the following dependency conflicts.

umap-learn 0.5.12 requires scikit-learn>=1.6, but you have scikit-learn 1.5.2 which is incompatible.

hdbscan 0.8.42 requires scikit-learn>=1.6, but you have scikit-learn 1.5.2 which is incompatible.

Mounted at /content/gdrive

You can now save your data files in: /content/gdrive/MyDrive/36106/assignment/AT3/data

Student Information

```
In [ ]: group_name = ""
student_name = "Yi An Tsai"
student_id = "25532767"
```

```
In [ ]: # Do not modify this code
print_tile(size="h1", key='group_name', value=group_name)
```

```
In [ ]: # Do not modify this code
print_tile(size="h1", key='student_name', value=student_name)
```

```
In [ ]: # Do not modify this code
print_tile(size="h1", key='student_id', value=student_id)
```

0. Python Packages

0.a Install Additional Packages

If you are using additional packages, you need to install them here using the command: `! pip install <package_name>`

```
In [ ]: !pip install "vegafusion[embed]>=1.5.0"
```

Collecting vegafusion>=1.5.0 (from vegafusion[embed]>=1.5.0)

Downloading vegafusion-2.0.3-cp39-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (1.1 kB)

Collecting arro3-core (from vegafusion>=1.5.0->vegafusion[embed]>=1.5.0)

Downloading arro3_core-0.8.0-cp311-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (515 bytes)

Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from vegafusion>=1.5.0->vegafusion[embed]>=1.5.0) (26.1)

Requirement already satisfied: narwhals>=1.42 in /usr/local/lib/python3.12/dist-packages (from vegafusion>=1.5.0->vegafusion[embed]>=1.5.0)

(2.20.0)

Downloading vegafusion-2.0.3-cp39-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (23.8 MB)

```
23.8/23.8 MB 38.4 MB/s eta 0:00:00
```

Downloading arro3_core-0.8.0-cp311-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (3.0 MB)

```
3.0/3.0 MB 56.7 MB/s eta 0:00:00
```

Installing collected packages: arro3-core, vegafusion

Successfully installed arro3-core-0.8.0 vegafusion-2.0.3

```
In [ ]: !pip install "vl-convert-python>=1.6.0"
```

Collecting vl-convert-python>=1.6.0

Downloading vl_convert_python-1.9.0.post1-cp37-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (5.1 kB)

Downloading vl_convert_python-1.9.0.post1-cp37-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (33.5 MB)

```
33.5/33.5 MB 59.9 MB/s eta 0:00:00
```

Installing collected packages: vl-convert-python

Successfully installed vl-convert-python-1.9.0.post1

0.b Import Packages

```
In [3]: import pandas as pd
import altair as alt
import numpy as np
```

B. Data Understanding

```
In [4]: # Do not modify this code
try:
    df = pd.read_csv(at.folder_path / "sales_order_header.csv")
except Exception as e:
    print(e)
```

B.1 Explore Dataset

```
In [ ]: df.head()
```

Out []:

	sales_order_id	revision_number	status	online_order_flag	customer_id	sales_person_id	territory_id	currency_rate_id	order_date	due_date	ship_d
0	1b9285b8-4501-4c92-adf7-ee74b4fa3d62	8.0	5.0	0.0	038a7b5e-5d15-4a1a-8305-e18131e402cc	28141e19-49e8-4090-9196-6e8f43b10de8	1e179cbb-7db9-4a66-aa94-e6fb9b3d613e	NaN	2011-05-30 22:00:00	2011-06-12 00:00:00	2011-06-12 00:00:00
1	b28eec58-f64d-4211-b604-bc9e801fb523	8.0	5.0	0.0	75c8fd92-2eb9-4223-b92c-f7c66751ed52	28141e19-49e8-4090-9196-6e8f43b10de8	1e179cbb-7db9-4a66-aa94-e6fb9b3d613e	NaN	2011-05-30 22:00:00	2011-06-12 00:00:00	2011-06-12 00:00:00
2	664be852-4095-4495-887a-4fc24bb299e6	8.0	5.0	0.0	e5107fe5-b23a-46a4-8e49-bd195ef3b713	27d62980-8d18-4170-95a5-7f7bb9cada5a	c8b914d0-8c4c-498f-af9f-6954c90f45db	4.0	2011-05-30 22:00:00	2011-06-12 00:00:00	2011-06-12 00:00:00
3	a3a724bb-b604-40f8-b4d2-30621d551610	8.0	5.0	0.0	cb04c96c-e647-4fe1-9fbd-7010a3e3049f	27d62980-8d18-4170-95a5-7f7bb9cada5a	c8b914d0-8c4c-498f-af9f-6954c90f45db	4.0	2011-05-30 22:00:00	2011-06-12 00:00:00	2011-06-12 00:00:00
4	ed512092-ed9b-4614-b6d0-80a340884550	8.0	5.0	0.0	274634cb-6c91-4372-a64e-cc4277a73779	16eb6889-25f7-475a-9eff-92024a00fde9	5f568b38-d738-4b7a-a27b-bc975b9084a2	NaN	2011-05-30 22:00:00	2011-06-12 00:00:00	2011-06-12 00:00:00

```
In [ ]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 31465 entries, 0 to 31464
Data columns (total 17 columns):
#   Column                Non-Null Count  Dtype
---  -
0   sales_order_id         31465 non-null  object
1   revision_number        31465 non-null  float64
2   status                 31465 non-null  float64
3   online_order_flag      31465 non-null  float64
4   customer_id            31465 non-null  object
5   sales_person_id        3806 non-null   object
6   territory_id           31465 non-null  object
7   currency_rate_id       13976 non-null  float64
8   order_date             31465 non-null  object
9   due_date               31465 non-null  object
10  ship_date              31465 non-null  object
11  sales_order_number     31465 non-null  object
12  account_number         31465 non-null  object
13  sub_total              31465 non-null  float64
14  tax_amount             31465 non-null  float64
15  freight                31465 non-null  float64
16  total_due              31465 non-null  float64
dtypes: float64(8), object(9)
memory usage: 4.1+ MB
```

```
In [ ]: df.shape
```

Out []: (31465, 17)

```
In [ ]: df.describe()
```

Out []:

	revision_number	status	online_order_flag	currency_rate_id	sub_total	tax_amount	freight	total_due
count	31465.000000	31465.0	31465.000000	13976.000000	31465.000000	31465.000000	31465.000000	31465.000000
mean	8.000953	5.0	0.879040	9191.499571	3491.065673	323.755743	101.173693	3915.995109
std	0.030864	0.0	0.326086	2945.170095	11093.452536	1085.054180	339.079427	12515.462713
min	8.000000	5.0	0.000000	2.000000	1.374000	0.109900	0.034400	1.518300
25%	8.000000	5.0	1.000000	8510.000000	56.970000	4.557600	1.424300	62.951900
50%	8.000000	5.0	1.000000	10074.000000	782.990000	62.639200	19.574800	865.204000
75%	8.000000	5.0	1.000000	11282.000000	2366.960000	189.597600	59.249300	2615.490800
max	9.000000	5.0	1.000000	12431.000000	163930.394300	17948.518600	5608.912100	187487.825000

```
In [ ]: df['sales_person_id'].isnull().sum()
```

Out[]: np.int64(27659)

In []: `df['currency_rate_id'].isnull().sum()`

Out[]: np.int64(17489)

In [5]: `# print the unique values of each column`
`for i in df.columns:`
 `print(f"Number of unique values in {i} : {len(df[i].unique())}")`

Number of unique values in sales_order_id : 31465
 Number of unique values in revision_number : 2
 Number of unique values in status : 1
 Number of unique values in online_order_flag : 2
 Number of unique values in customer_id : 19119
 Number of unique values in sales_person_id : 18
 Number of unique values in territory_id : 10
 Number of unique values in currency_rate_id : 2515
 Number of unique values in order_date : 1124
 Number of unique values in due_date : 1124
 Number of unique values in ship_date : 1124
 Number of unique values in sales_order_number : 31465
 Number of unique values in account_number : 19119
 Number of unique values in sub_total : 4747
 Number of unique values in tax_amount : 4745
 Number of unique values in freight : 4744
 Number of unique values in total_due : 4754

In []: `# the possible values in this columns`
`df['status'].value_counts()`

Out[]:

	count
status	
5.0	31465

dtype: int64

In []: `# the possible values in this columns`
`df['online_order_flag'].value_counts()`

Out[]:

	count
online_order_flag	
1.0	27659
0.0	3806

dtype: int64

In [6]: `df['online_order_flag'].value_counts(normalize= 'index')`

Out[6]:

	proportion
online_order_flag	
1.0	0.87904
0.0	0.12096

dtype: float64

In []: `# convert the object to datetime`
`df['order_date'] = pd.to_datetime(df['order_date'])`
`df['due_date'] = pd.to_datetime(df['due_date'])`
`df['ship_date'] = pd.to_datetime(df['ship_date'])`

`# check if any items is shipped after due date.`
`df[df['ship_date'] > df['due_date']]`

Out[]:

sales_order_id	revision_number	status	online_order_flag	customer_id	sales_person_id	territory_id	currency_rate_id	order_date	due_date	ship_date
----------------	-----------------	--------	-------------------	-------------	-----------------	--------------	------------------	------------	----------	-----------



In []: `dataset_insights = ""`
 There are 31465 records and 17 columns in this dataset, while the 'sales_person_id' and 'currency_rate_id' columns have missing values, at 27769 and 17489 respectively.

 Three columns (customer_id, 'territory_id' and 'account_number') are the same as last dataset(customer.csv).

 The meaning of the following columns:
 1. revision_number: the times each order is revised
 2. status: order status, all orders are in status 5.
 3. online_order_flag: check if it it order online, 1 = yes, 0 = no, most of them are from online.
 4. currency_rate_id: the order need to exchange currency, nan means use local currency.
 5. sub_total: the amount after discount
 6. tax_mat = the amount of tax
 7. freight: the shipping fee.
 8. total_due: the total amount of order. (sub_total + tax_amount + freight = total_due)

No order is shipped after due date.
Merely 12% data are off line order.
"" ""

"" ""

```
In [ ]: # Do not modify this code
print_tile(size="h3", key='dataset_insights', value=dataset_insights)
```

B.2 Explore Feature of Interest revision_number

```
In [ ]: df['revision_number'].unique()
```

Out[]: array([8., 9.])

```
In [ ]: df['revision_number'].value_counts()
```

	count
revision_number	
8.0	31435
9.0	30

dtype: int64

```
In [ ]: df_rev9 = df[df['revision_number'] == 9]
df_rev9.head()
```

	sales_order_id	revision_number	status	online_order_flag	customer_id	sales_person_id	territory_id	currency_rate_id	order_date	due_date	shi
7466	9807c495-3782-42df-ba25-087d8856321a	9.0	5.0	0.0	cedfdc6e-dfa5-44de-9096-4a22e9c34958	9737ee67-ea97-4ab7-9e6c-70f8878c6ab2	25d73ad2-9e13-410b-8b0d-5f26e2b9e4f2	8404.0	2013-05-29 22:00:00	2013-06-11 00:00:00	20
7512	d71412ab-f22a-4ff4-bc88-e057c9354db0	9.0	5.0	0.0	137bb03e-368e-4cbb-9804-0e9f745350f4	9737ee67-ea97-4ab7-9e6c-70f8878c6ab2	25d73ad2-9e13-410b-8b0d-5f26e2b9e4f2	8404.0	2013-05-29 22:00:00	2013-06-11 00:00:00	20
7513	c5ccc270-60ac-48c2-95a3-726474ae1462	9.0	5.0	0.0	35882dd2-4308-4903-be85-a05bdbc8ed5	9737ee67-ea97-4ab7-9e6c-70f8878c6ab2	25d73ad2-9e13-410b-8b0d-5f26e2b9e4f2	8404.0	2013-05-29 22:00:00	2013-06-11 00:00:00	20
8036	e43ec355-5d08-468a-8d7c-68768f71487f	9.0	5.0	0.0	5e16c2d2-1652-403f-8bbb-c61331ac143d	9737ee67-ea97-4ab7-9e6c-70f8878c6ab2	25d73ad2-9e13-410b-8b0d-5f26e2b9e4f2	8745.0	2013-06-29 22:00:00	2013-07-12 00:00:00	20
8091	0c3fa516-2ec6-4b10-9715-9240a5404bc9	9.0	5.0	0.0	f205f927-8c9c-4db4-bd0e-93bcd773b778	9737ee67-ea97-4ab7-9e6c-70f8878c6ab2	25d73ad2-9e13-410b-8b0d-5f26e2b9e4f2	8745.0	2013-06-29 22:00:00	2013-07-12 00:00:00	20

```
In [ ]: df_rev9.describe(include='all')
```

	sales_order_id	revision_number	status	online_order_flag	customer_id	sales_person_id	territory_id	currency_rate_id	order_date	due_date	sl
count	30	30.0	30.0	30.0	30	30	30	29.000000	30	30	
unique	30	NaN	NaN	NaN	10	2	1	NaN	12	12	
top	9807c495-3782-42df-ba25-087d8856321a	NaN	NaN	NaN	cedfdc6e-dfa5-44de-9096-4a22e9c34958	9737ee67-ea97-4ab7-9e6c-70f8878c6ab2	25d73ad2-9e13-410b-8b0d-5f26e2b9e4f2	NaN	2013-11-29 23:00:00	2013-12-12 00:00:00	20
freq	1	NaN	NaN	NaN	4	29	30	NaN	4	4	
mean	NaN	9.0	5.0	0.0	NaN	NaN	NaN	10084.724138	NaN	NaN	
std	NaN	0.0	0.0	0.0	NaN	NaN	NaN	1158.031794	NaN	NaN	
min	NaN	9.0	5.0	0.0	NaN	NaN	NaN	8404.000000	NaN	NaN	
25%	NaN	9.0	5.0	0.0	NaN	NaN	NaN	9086.000000	NaN	NaN	
50%	NaN	9.0	5.0	0.0	NaN	NaN	NaN	10087.000000	NaN	NaN	
75%	NaN	9.0	5.0	0.0	NaN	NaN	NaN	11077.000000	NaN	NaN	
max	NaN	9.0	5.0	0.0	NaN	NaN	NaN	12089.000000	NaN	NaN	

```
In [ ]: feature_1_insights = """
There are only two categories in this column-8 and 9.
Most records are category 8 (31,435 rows), and only 30 are category 9.
The 30 category-9 rows all share the same territory_id, are offline, have status = 5, and 29 come from the same customer.
This strongly indicates that they form a special batch, likely from a data import or correction.
```

"""

```
In [ ]: # Do not modify this code
print_tile(size="h3", key='feature_1_insights', value=feature_1_insights)
```

B.3 Explore Feature of Interest territory_id

```
In [ ]: len(df['territory_id'].unique())
```

```
Out[ ]: 10
```

```
In [ ]: df['territory_id'].value_counts()
```

```
Out[ ]:
```

	count
territory_id	
2ac923e9-3043-4f5e-bae0-ad28089bf187	6843
5f568b38-d738-4b7a-a27b-bc975b9084a2	6224
d90fce1e-44e0-4d8f-8125-2f9ba1d18cbc	4594
c8b914d0-8c4c-498f-af9f-6954c90f45db	4067
e0e3bac4-790e-4617-84b3-be0c2bdb7070	3219
0ad4c625-bb65-4376-8a41-0a65719b0db8	2672
25d73ad2-9e13-410b-8b0d-5f26e2b9e4f2	2623
1e179cbb-7db9-4a66-aa94-e6fb9b3d613e	486
5b963d6f-3cda-4843-ade2-0cb22c0eccaf	385
7559c9bb-7906-435e-934d-82cfee38295e	352

dtype: int64

```
In [ ]: df_avg_due = df.groupby('territory_id')['total_due'].mean().reset_index()
df_avg_due_top10 = df_avg_due.sort_values(by='total_due', ascending=False).head(10)

# Create a mapping from original territory_id to numerical IDs
territory_id_mapping = {id: i + 1 for i, id in enumerate(df_avg_due_top10['territory_id'])}

# Apply the mapping to the DataFrame
df_avg_due_top10['territory_numerical_id'] = df_avg_due_top10['territory_id'].map(territory_id_mapping)

display(df_avg_due_top10.head(10))
```

	territory_id	total_due	territory_numerical_id
4	5b963d6f-3cda-4843-ade2-0cb22c0eccaf	23151.426616	1
6	7559c9bb-7906-435e-934d-82cfee38295e	22216.504626	2
1	1e179cbb-7db9-4a66-aa94-e6fb9b3d613e	18280.039850	3
7	c8b914d0-8c4c-498f-af9f-6954c90f45db	4523.956033	4
5	5f568b38-d738-4b7a-a27b-bc975b9084a2	4362.242061	5
8	d90fce1e-44e0-4d8f-8125-2f9ba1d18cbc	3931.576049	6
0	0ad4c625-bb65-4376-8a41-0a65719b0db8	3038.828348	7
9	e0e3bac4-790e-4617-84b3-be0c2bdb7070	2663.575243	8
2	25d73ad2-9e13-410b-8b0d-5f26e2b9e4f2	2089.142042	9
3	2ac923e9-3043-4f5e-bae0-ad28089bf187	1726.490734	10

```
In [ ]: # Chart for Average Total Due (Bars)
bars = alt.Chart(df_avg_due_top10).mark_bar(opacity=0.7, color='lightblue').encode(
    x=alt.X('territory_numerical_id:0', title='Territory ID (Numerical)'),
    y=alt.Y('total_due', axis=alt.Axis(title='Average Total Due', titleColor='lightblue')),
    tooltip=['territory_id', 'territory_numerical_id', 'total_due']
)

# Filter the original DataFrame to include only the top 10 territory_ids
top10_territory_ids = df_avg_due_top10['territory_id'].tolist()
df_filtered_top10_territories = df[df['territory_id'].isin(top10_territory_ids)].copy()

# Map the territory_id in the filtered DataFrame to numerical IDs
df_filtered_top10_territories['territory_numerical_id'] = df_filtered_top10_territories['territory_id'].map(territory_id_mapping)

# Calculate the count of orders for each numerical territory ID
territory_counts = df_filtered_top10_territories['territory_numerical_id'].value_counts().reset_index()
territory_counts.columns = ['territory_numerical_id', 'count']
territory_counts = territory_counts.sort_values(by='territory_numerical_id') # Sort by numerical ID for consistent line chart order

# Counts
lines = alt.Chart(territory_counts).mark_line(point=True, color='steelblue').encode(
    x=alt.X('territory_numerical_id:0', title='Territory ID (Numerical)'),
    y=alt.Y('count', axis=alt.Axis(title='Total Count of Orders', titleColor='steelblue')),
```

```

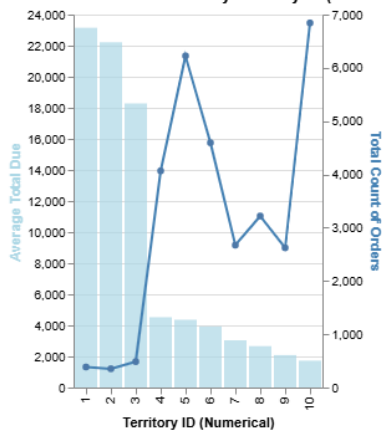
    tooltip=['territory_numerical_id', 'count']
)

# Layer the two charts and resolve scales for independent Y-axes
combined_layered_chart = alt.layer(bars, lines).resolve_scale(
    y='independent'
).properties(
    title='Average Total Due and Order Counts by Territory ID (Numerical Labels)'
)

combined_layered_chart.display()

```

Average Total Due and Order Counts by Territory ID (Numerical Labels) ...



```

In [ ]: feature_2_insights = """
The territory_id has 10 unique values, and a chart showing the average total_due for each of these territories has been generated.
To simplify analysis and plotting, these UUIDs have been mapped to numerical IDs from 1 to 10.
The top 3 territories with the highest average total due are represented by numerical IDs 1, 2, and 3.

In terms of order count, the top 3 territories are represented by numerical IDs 10, 5, and 6 respectively.
This finding reveals that order volume and average total due show opposite trends.
"""

```

```

In [ ]: # Do not modify this code
print_tile(size="h3", key='feature_2_insights', value=feature_2_insights)

```

B.4 Explore Feature of Interest `total_due`

```

In [ ]: df['total_due'].head()

```

```

Out[ ]:
   total_due
0  23153.2339
1   1457.3288
2  36865.8012
3  32474.9324
4   472.3108

```

dtype: float64

```

In [ ]: import altair as alt
alt.data_transformers.disable_max_rows()
alt.data_transformers.enable('default')
alt.data_transformers.enable("vegafusion")

```

```

Out[ ]: DataTransformerRegistry.enable('vegafusion')

```

```

In [ ]: df['total_due'].describe() #most value fall between $62~ $2615, but max is extremely large.

```

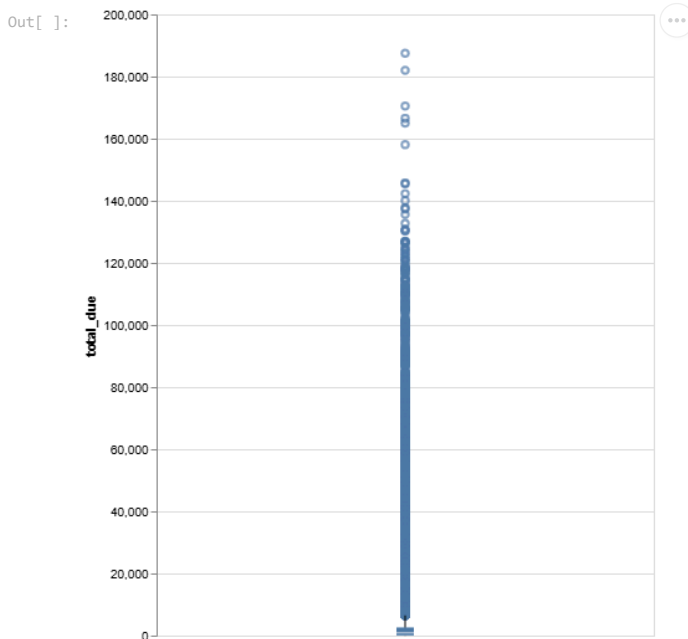
```

Out[ ]:
   total_due
count  31465.000000
mean    3915.995109
std    12515.462713
min       1.518300
25%     62.951900
50%    865.204000
75%   2615.490800
max  187487.825000

```

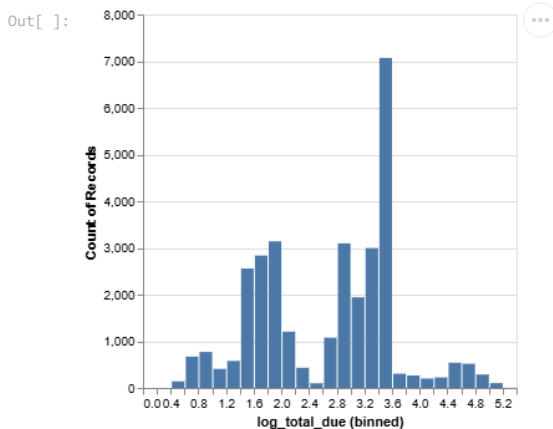
dtype: float64

```
In [ ]: # boxplot of total_due, numerous outliers (aligned with last cell)
alt.Chart(df).mark_boxplot().encode(
    y='total_due'
).properties(
    width=400,
    height=500
)
```



```
In [ ]: df_log = df[df['total_due'] > 0].copy()
df_log['log_total_due'] = np.log10(df['total_due']) # applied log10 to observe the distribution

# barchart
alt.Chart(df_log).mark_bar().encode(
    alt.X('log_total_due:Q', bin=alt.Bin(maxbins=50)),
    y='count()'
)
```



```
In [ ]: feature_n_insights = """
Over 50% of the data falls between $62 ~ $2615, but the max is extremely large, indicating numerous outliers.

To better observe the distribution, log10 is applied to this feature, revealing two main customer groups with different spending habits and
Clustering or classification might be applied in this direction.
"""
```

```
In [ ]: # Do not modify this code
print_tile(size="h3", key='feature_n_insights', value=feature_n_insights)
```

```
In [ ]:
```

```
In [ ]:
```